

Simon says game

Lisa Werner

Studenten Nr.: 0239544

Noorderpoort

25998 – Software developer

KD: Microcontrollers

Docent: Niels Vermeulen

13.06.2025

Index

Project voorstel.....	3
Doel van het project.....	3
Waarom dit project.....	3
Benodigdheden.....	3
Repository link en inhoud.....	4
Procesverslag.....	5
Planning.....	5
Prototype bouwen.....	6
Reflectie (Korthagen Reflectiemodel).....	7
Conclusie.....	9

Project voorstel

Doel van het project

Ik heb ervoor gekozen om het klassieke geheugenspel Simon Says (elektronische versie) na te maken met een Arduino. Het spel laat willekeurige licht- en geluidspatronen zien die de speler moet nadoen. Er komt elke ronde een nieuw licht en geluid bij de al bestaande sequentie. Bij een fout begint het spel opnieuw.

Waarom dit project

Ik kies dit project omdat het een leuke en interactieve manier is om te leren werken met LEDs, buzzers, knoppen en logica in een Arduino. Het is uitdagend, maar goed uitvoerbaar binnen de tijd van het keuzedeel.

Benodigdheden

Component	Aantal	Notities
Arduino Uno	1	
12 mm push button	4	
LEDs	4	Rood, Blauw, Geel, Groen
Piezo buzzer	1	
Weerstanden	4	220 Ohm

Repository link en inhoud

[GitLab Repository link](#)

Inhoud

- code voor het Simon says spel
- pitches.h sketch
- code voor de test sequentie
- documentatie van mijn project
- fritzing schema met alle componenten voor het simon says speel
- foto's en video's van het werkende project

Procesverslag

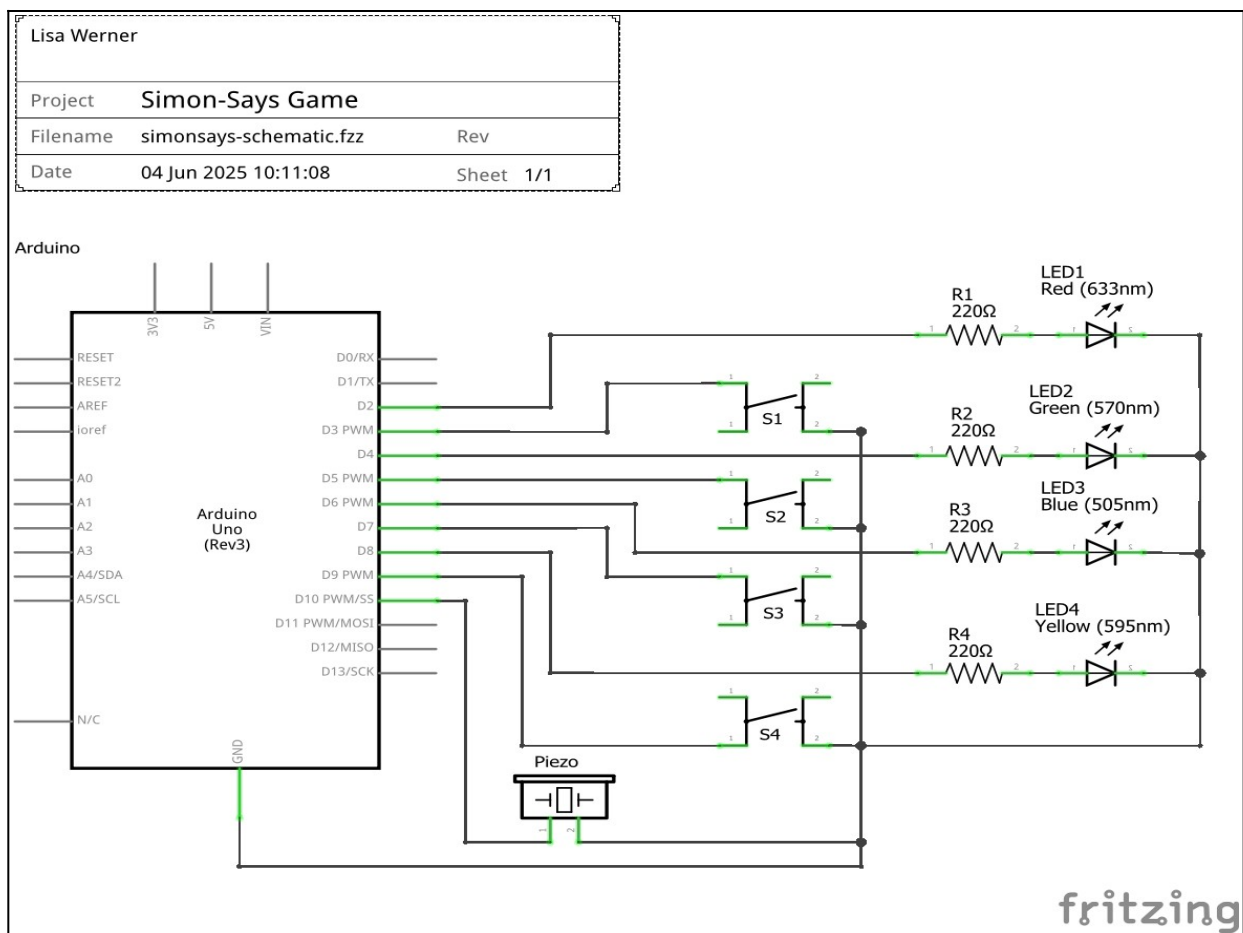
Planning

- Gameloop:

Het spel laat een bepaalde toon horen, waarbij de overeenkomstige drukknop oplicht. De speler dient binnen een bepaalde tijd die knop in te drukken. Indien correct, hoort de speler deze toon opnieuw, gevolgd door een tweede toon. De speler dient dan de 2 overeenkomstige knoppen terug in te drukken. Vervolgens laat het spel 3 tonen horen, enz.

Dit blijft zo verdergaan tot de speler een fout maakt of te lang wacht.

- Elektronica schema



Prototype bouwen

1 Alle componenten volgens het schema op het breadboard zetten

2 Componenten testen:

Voordat ik met coderen bezig ging, heb ik mijn componenten op het breadboard aangesloten en een kleine testsequentie geschreven om te testen of alle componenten werken zoals het volgens het schema hoort. Voor de pushbuttons heb ik ervoor gekozen om de interne Arduino pullup resistor van de digital pins te gebruiken.

3 Code:

3.1 Licht sequentie genereren en opslaan

De eerste stap was de lichtsequentie voor het spel te genereren. Omdat arrays in Arduino/C++ een vaste lengte hebben, heb ik de keuze gemaakt om mijn max aantal rondes aan de lengte van de array te koppelen. Om een willkeurige sequentie te genereren maak ik gebruik van randomSeed. Het initialiseert de pseudo-willekeurige getallengenerator, waardoor deze op een willekeurig punt in de willekeurige reeks start.

3.2 Geluid

Daarna heb ik de sketch 'pitches.h' toegevoegd aan mijn project. Nadat dit opgezet was heb ik een aantal klanken en methodes gedefinieerd zodat ik er later makkelijk met mijn code bij kan.

Vervolgens heb ik dan het blinken van de LED's met het geluid gekoppeld.

3.3 Gebruiker input via de push buttons

De volgende stap was de input van de buttons uit te lezen en op basis daarvan te bepalen welke button er ingedrukt was. Ik heb ervoor gebruik gemaakt thread blocking while loops van, omdat ik wilde dat de volgende code alleen uitgevoerd wordt als er geldige input is. Nadat er valide input ingevoerd wordt, heb ik een check uitgevoerd om te kijken of de ingedrukte knop de juiste was.

3.4 Win of Lose conditions

Nu dat de input van de speler ingelezen en gechecked is kan ik via code bepalen of het goed of fout was. Als het de juiste knop was speelt een klein muziekje en wordt de sequentie uitgebreid en afgespeeld. Als het de foute knop was, speelt er een game-over muziekje en is het spel voorbij. Als de laatste ronde correct af is, speelt er een game-compleet muziekje.

4 Testing

De laatste stap was om mijn spel te testen om te kijken of het goed werkt, en dingen zoals de geluidjes en delays nog iets te tweaken.

Reflectie (Korthagen Reflectiemodel)

1. Met het project beginnen

Ik begon gewoon met coderen, maar de code werd al snel onoverzichtelijk en moeilijk uit te breiden. In het verleden had ik vaak last van “premature optimizing”, waardoor het te lang duurde voordat ik er echt begon met coderen, dus wilde ik deze keer proberen om gewoon meteen aan de slag te gaan.

Achteraf gezien was dat ook niet de ideale manier, omdat het zonder plan snel chaotisch werd en het moeilijk werd dingen achteraf toevoegen of aan te passen.

Het was belangrijk om een beter idee over de flow van de code te hebben voordat ik met programmeren bezig ging.

Ik had beter met een simple flowchart kunnen beginnen.

Volgende keer zal ik wel weer een planning maken, maar het simpel houden en als ik tegen iets aanloop de planning opnieuw bekijken en beoordelen.

2. Debouncing problemen met het lezen van de pushbutton input.

Tijdens het testen van de pushbutton kwam ik er achter dat er soms meerdere pushes gedetecteerd worden.

Achteraf gezien was het wel een vrij simpel probleem met een heel simpel oplossing. De loop draaide meerdere keren per seconde, dus de input was meerdere keren gedetecteerd.

Nadat ik een delay en een “was pressed” flag toegevoegd had, werkte het gewoon consistent.

Volgende keer ga ik gewoon direct een flag toevoegen om dit probleem te voorkomen.

3. Probleem als ik mijn sketch op de Arduino schrijf, speelt de eerste loop soms twee keer af.

Toen ik de sketch op mijn Arduino schreef, werd het eerste deel van de sequentie (de start en de lus) soms twee keer afgespeeld.

Ik dacht eerst dat er een fout in mijn code zat, maar ik vondt het wel raar dat het zo inconsistent was en als ik de arduino gewoon resette ging alles goed.

Na wat onderzoek kwam ik er achter dat de oorzaak ervan de Serial Monitor was. Als je een sketch upload terwijl de Serial Monitor open is, zorgt het ervoor dat het board één keer wordt uitgevoerd, en daarna gereset wordt en de nieuwe schets wordt uitgevoerd.

De oplossing ervoor was er een delay van 150 ms in de start functie te zetten. Daardoor reset het board gewoon voordat de code uitgevoerd wordt.

Dat kan ik volgende keer gewoon bijvoegen.

Conclusie

In dit project heb ik een werkende versie van het spel *Simon Says* na gebouwd met een Arduino.

Ik heb ervoor LEDs, een piezo buzzer en pushbuttons gebruikt. Het doel was om een spel te maken waarbij de speler een kleuren en tonen moet onthouden en correct nadoen. De sequentie wordt elke “ronde” eentje langer, het max is 100 rondes.

Tijdens het maken van het project heb ik veel geleerd over het structureren van code, het werken met een procedurele programmeertaal, en het werken met inputs op een microcontroller. Een van de grootste uitdagingen was het correct uitlezen van de pushbutton input zonder dat er de input soms meerdere keren geregistreerd werd. Dit heb ik opgelost door logica toe te voegen die alleen veranderingen in button status detecteert.

Het eindresultaat is een stabiel en goed werkend spel dat reageert zoals bedoeld. Ik ben tevreden met de werking en de opbouw van mijn code. Als ik het project zou uitbreiden of verbeteren, zou ik bijvoorbeeld een scoreweergave via een display toevoegen, of het spel moeilijker maken door het tempo geleidelijk te verhogen en de pushbuttons te vervangen met pushbuttons waar led al ingebouwt zit.

